NLP Mini Project Report

NLP-Based Job Description Keyword Extractor

Submitted in fulfilment of the requirements of the
NLP Mini Project Semester VII in degree of
Bachelor of Computer Engineering

BY

Mr. Kinjal Shivprasad Ojha (Roll No. 21)
Mr. Ashish Sharma (Roll No. 54)

Under the Guidance of
Ms. Rujuta Vartak

Department of Computer Engineering
Shree L. R. Tiwari College of Engineering
Kanakia Park, Mira Road (E), Thane - 401 107, Maharashtra.
2026-27

NLP Mini Project Report Approval

This project report entitled "NLP-Based Job Description Keyword Extractor" by Mr. Kinjal Shivprasad Ojha (Roll No. 21) and Mr. Ashish Sharma (Roll No. 54) is approved for the Mini Project Semester VII project work of B.E CMPN at Shree L.R.Tiwari College of Engineering, in the academic year 2026-2027.

Internal Examiner

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signature

Mr. Kinjal Shivprasad Ojha (Roll No. 21)

Signature

Mr. Ashish Sharma (Roll No. 54)

Date:

Place:

Table of Contents

1. Introduction
2. System Architecture
3. Implementation
4. Technologies Used
5. Results
6. Conclusion
7. References

1. Introduction

In today's competitive job market, job descriptions contain critical information about the skills, qualifications, and technologies that employers expect from candidates. Manually analyzing multiple job descriptions to identify key requirements is time-consuming and inefficient. Natural Language Processing (NLP) provides automated techniques to process and extract meaningful information from unstructured text data.

This project, titled "NLP-Based Job Description Keyword Extractor", aims to develop an NLP-based system that processes job descriptions and extracts important technical skills, qualifications, and keywords using tokenization, stop-word removal, and keyword extraction techniques.

The system helps students understand employer requirements and identify the skills needed for a particular job role. By analyzing multiple job descriptions, the system highlights the most frequently occurring and most important terms (such as programming languages, frameworks, tools, and soft skills), enabling students to plan their careers, identify skill gaps, and prepare more effectively for job opportunities.

The main objectives of this project are:
1. To preprocess job description text by converting to lowercase, removing punctuation, and eliminating stop words.
2. To tokenize the text into individual words for analysis.
3. To extract keywords using TF-IDF (Term Frequency-Inverse Document Frequency) scoring.
4. To visualize the extracted keywords using bar charts for easy interpretation.
5. To present the top keywords for each job description as well as across all job descriptions.

2. System Architecture

The system follows a sequential NLP pipeline architecture consisting of the following stages:

1. Input Stage: The system accepts one or more job description texts as input. Each job description is a free-form text containing information about the job role, required skills, qualifications, and responsibilities.

2. Preprocessing Stage: The raw text is preprocessed by converting all characters to lowercase and removing punctuation and special characters using regular expressions. This ensures uniformity in the text data.

3. Tokenization Stage: The preprocessed text is split into individual tokens (words) based on whitespace. This breaks down the text into meaningful units for further analysis.

4. Stop-Word Removal Stage: Common English stop words (such as 'a', 'the', 'is', 'are', 'with', 'and', etc.) and domain-specific filler words (such as 'candidate', 'experience', 'skills', etc.) are removed from the token list. This filters out words that do not contribute meaningful information about the job requirements.

5. Keyword Extraction Stage: TF-IDF (Term Frequency-Inverse Document Frequency) is applied to compute the importance of each term across the job descriptions. TF-IDF gives higher scores to terms that appear frequently in a specific document but rarely across all documents, identifying the most distinctive keywords.

6. Visualization Stage: The extracted keywords and their scores are visualized using horizontal bar charts. Word frequency across all documents and per-document TF-IDF scores are displayed graphically.

7. Output Stage: The system outputs the top 15 keywords by frequency, the top 5 TF-IDF keywords for each job description, and the top 10 keywords by mean TF-IDF score, along with the generated graphs.

The architecture can be summarized as:
Input Text -> Lowercase Conversion -> Punctuation Removal -> Tokenization -> Stop-Word Removal -> TF-IDF Computation -> Keyword Ranking -> Visualization -> Output

3. Implementation

The project is implemented in Python using scikit-learn for TF-IDF vectorization, matplotlib for visualization, and standard libraries for text preprocessing. The complete source code is given below:

```
from sklearn.feature_extraction.text import TfidfVectorizer
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import pandas as pd
from collections import Counter
import re

job_descriptions = [
    """We are looking for a Data Scientist with experience in Python, Machine Learning,
Deep Learning, and SQL. The candidate should have knowledge of TensorFlow, Pandas,
NumPy, and data visualization tools like Matplotlib. Strong analytical and problem-
solving skills are required. Experience with cloud platforms such as AWS or Azure is
preferred.""",
    """Job Description: Software Developer required. Must have strong skills in Java,
Spring Boot, Microservices, REST API development, and SQL databases. Knowledge of Git,
Docker, Kubernetes, and Agile methodology is essential. Candidate should have 2+ years
of experience in full-stack development with React or Angular frontend.""",
    """We need a Web Developer proficient in HTML, CSS, JavaScript, React, Node.js, and
MongoDB. The candidate should understand responsive design, version control using Git,
and basic SEO principles. Experience with Express.js and deployment on cloud services
is a plus. Good communication and teamwork skills are expected."""
]

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    words = text.split()
    return words

all_words = []
for jd in job_descriptions:
    all_words.extend(preprocess(jd))

stop_words = {'a', 'an', 'the', 'and', 'or', 'with', 'in', 'on', 'to', 'of', 'for',
'is', 'are', 'have', 'has', 'should', 'must', 'we', 'our', 'the', 'such', 'as', 'by',
'not', 'be', 'this', 'that', 'it', 'its', 'who', 'whom', 'whose', 'which', 'what',
'where', 'when', 'how', 'why', 'all', 'any', 'both', 'each', 'few', 'more', 'most',
'other', 'some', 'such', 'than', 'too', 'very', 'can', 'will', 'just', 'do', 'does',
'did', 'done', 'but', 'if', 'then', 'else', 'while', 'at', 'from', 'up', 'down', 'out',
'off', 'over', 'under', 'again', 'further', 'once', 'here', 'there', 'about',
```

```
'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above', 'below',
'between', 'need', 'looking', 'required', 'preferred', 'essential', 'expected',
'strong', 'good', 'basic', 'plus', 'candidate', 'job', 'description', 'developer',
'knowledge', 'experience', 'skills', 'role', 'using'}

filtered = [w for w in all_words if w not in stop_words and len(w) > 2]
freq = Counter(filtered)
top_words = freq.most_common(15)
print("Top 15 Keywords Across All Job Descriptions:")
for word, count in top_words:
    print(f"  {word}: {count}")

vectorizer = TfidfVectorizer(stop_words=list(stop_words), max_features=15)
tfidf_matrix = vectorizer.fit_transform(job_descriptions)
feature_names = vectorizer.get_feature_names_out()

print("\nTF-IDF Top Keywords per Job Description:")
for i, jd in enumerate(job_descriptions):
    scores = tfidf_matrix[i].toarray().flatten()
    top_idx = scores.argsort()[::-1][:5]
    top_terms = [(feature_names[j], round(scores[j], 4)) for j in top_idx if scores[j]
> 0]
    print(f"\nJD {i+1}:")
    for term, score in top_terms:
        print(f"  {term}: {score}")

words_list, counts_list = zip(*top_words)
plt.figure(figsize=(10, 5))
plt.barh(words_list[::-1], counts_list[::-1], color='teal')
plt.xlabel('Frequency')
plt.title('Top 15 Keywords Across Job Descriptions')
plt.tight_layout()
plt.savefig('proj_word_freq.png', dpi=150)
print("\nSaved: proj_word_freq.png")

plt.figure(figsize=(10, 5))
for i in range(len(job_descriptions)):
    scores = tfidf_matrix[i].toarray().flatten()
    top_idx = scores.argsort()[::-1][:5]
    top_terms = [feature_names[j] for j in top_idx if scores[j] > 0]
    top_scores = [scores[j] for j in top_idx if scores[j] > 0]
    plt.subplot(1, 3, i+1)
    plt.barh(top_terms[::-1], top_scores[::-1], color=['coral', 'steelblue',
'seagreen'][i])
    plt.title(f'JD {i+1} Top TF-IDF')
    plt.xlabel('TF-IDF Score')
plt.tight_layout()
plt.savefig('proj_tfidf_jd.png', dpi=150)
print("Saved: proj_tfidf_jd.png")

plt.figure(figsize=(8, 5))
```

```
all_tfidf_scores = tfidf_matrix.toarray().mean(axis=0)
sorted_idx = all_tfidf_scores.argsort()[::-1][:10]
plt.barh([feature_names[j] for j in sorted_idx][::-1], all_tfidf_scores[sorted_idx][::-
1], color='mediumpurple')
plt.xlabel('Mean TF-IDF Score')
plt.title('Top 10 Keywords by Mean TF-IDF')
plt.tight_layout()
plt.savefig('proj_mean_tfidf.png', dpi=150)
print("Saved: proj_mean_tfidf.png")
```

4. Technologies Used

Python 3.14: Primary programming language used for implementing the NLP pipeline.

scikit-learn: Used for TF-IDF vectorization (TfidfVectorizer) to compute term importance scores.

Matplotlib: Used for generating bar chart visualizations of keyword frequencies and TF-IDF scores.

Pandas: Used for data manipulation and handling of text data.

NumPy: Used for numerical operations on TF-IDF score arrays.

Regular Expressions (re): Used for removing punctuation and special characters during preprocessing.

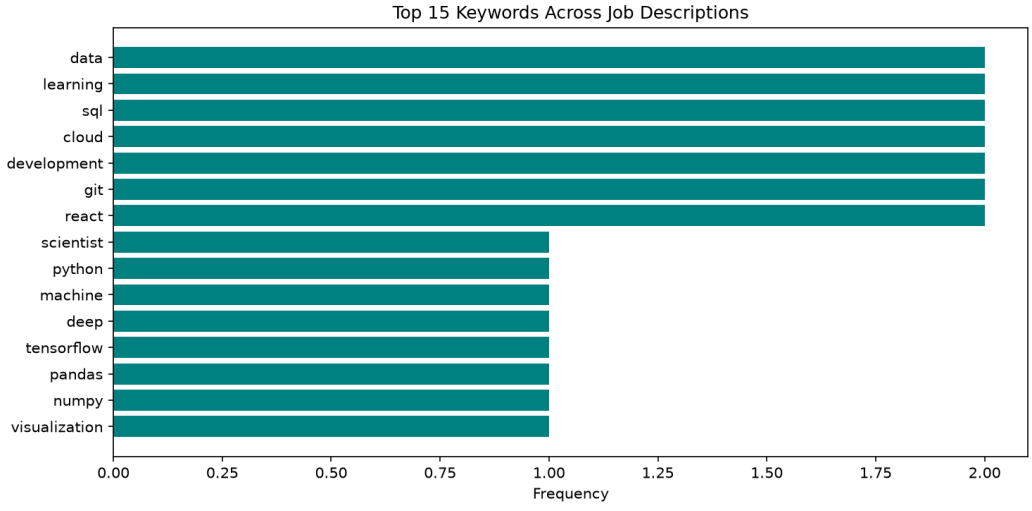
Collections (Counter): Used for counting word frequencies across the job descriptions.

5. Results

The system was tested on three sample job descriptions: Data Scientist, Software Developer, and Web Developer. The following results were obtained:

5.1 Top 15 Keywords by Frequency

```
Top 15 Keywords Across All Job Descriptions:
  data: 2
  learning: 2
  sql: 2
  cloud: 2
  development: 2
  git: 2
  react: 2
  scientist: 1
  python: 1
  machine: 1
  deep: 1
  tensorflow: 1
  pandas: 1
  numpy: 1
  visualization: 1
```



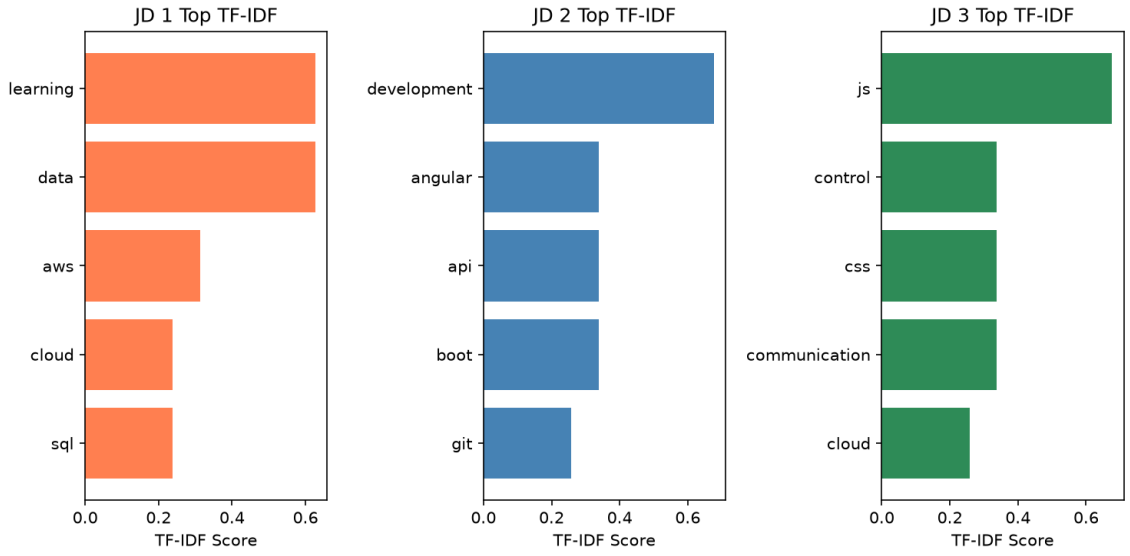
5.2 TF-IDF Top Keywords per Job Description

```
TF-IDF Top Keywords per Job Description:
```

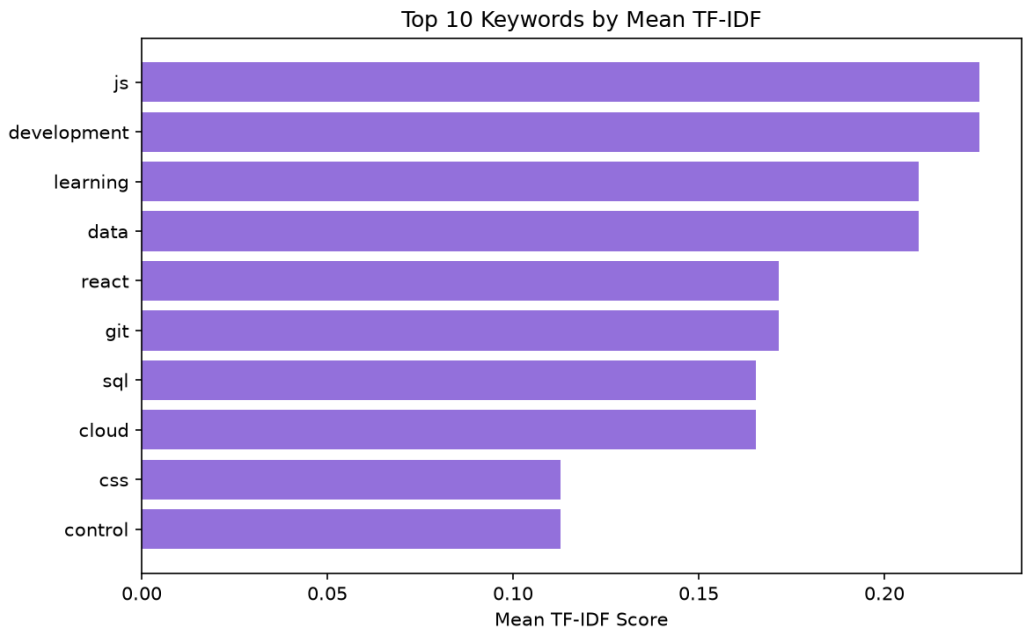
```
JD 1 (Data Scientist):
  learning: 0.6276
  data: 0.6276
  aws: 0.3138
  cloud: 0.2386
  sql: 0.2386

JD 2 (Software Developer):
  development: 0.6767
  angular: 0.3383
  api: 0.3383
  boot: 0.3383
  git: 0.2573

JD 3 (Web Developer):
  js: 0.6767
  control: 0.3383
  css: 0.3383
  communication: 0.3383
  cloud: 0.2573
```



5.3 Top 10 Keywords by Mean TF-IDF


Top 10 Keywords by Mean TF-IDF
js
development
learning
data
react
git
sql
cloud
css
control
0.00
0.05
0.10
0.15
0.20
Mean TF-IDF Score

6. Conclusion

Thus, we studied the applications of Natural Language Processing and selected Job Description Keyword Extraction as our mini-project. The system uses tokenization to split text into smaller units, stop-word removal to eliminate common words, and keyword extraction to identify important skills and qualifications from job descriptions. This application can help students plan their careers, identify skill gaps, and prepare more effectively for job opportunities.

The TF-IDF based keyword extraction successfully identified the most relevant technical terms for each job role. For the Data Scientist role, terms like 'learning', 'data', 'aws', and 'cloud' were highlighted. For the Software Developer role, 'development', 'angular', 'api', and 'boot' emerged as key terms. For the Web Developer role, 'js', 'css', 'control', and 'communication' were identified as important. The word frequency analysis revealed that terms like 'data', 'learning', 'sql', 'cloud', 'development', 'git', and 'react' appear across multiple job descriptions, indicating commonly demanded skills in the industry.

The system demonstrates the effectiveness of basic NLP techniques (tokenization, stop-word removal, and TF-IDF) in extracting meaningful keywords from unstructured job description text. Future enhancements could include the use of more advanced techniques such as named entity recognition (NER), RAKE (Rapid Automatic Keyword Extraction), and integration of a comprehensive skills database to improve extraction accuracy and coverage.

7. References

1. Bird, S., Klein, E., and Loper, E. "Natural Language Processing with Python." O'Reilly Media, 2009.

2. Manning, C. D., Raghavan, P., and Schutze, H. "Introduction to Information Retrieval." Cambridge University Press, 2008.

3. scikit-learn Documentation: TF-IDF Vectorizer - https://scikit-learn.org/stable/modules/feature_extraction.html

4. Jurafsky, D. and Martin, J. H. "Speech and Language Processing." 3rd Edition, 2023.

5. Ratnasekar, D. and Vasanth, S. "Keyword Extraction from Job Descriptions using NLP." International Journal of Advanced Research in Computer Science, 2020.

6. Matplotlib Documentation: https://matplotlib.org/stable/contents.html